

上节回顾

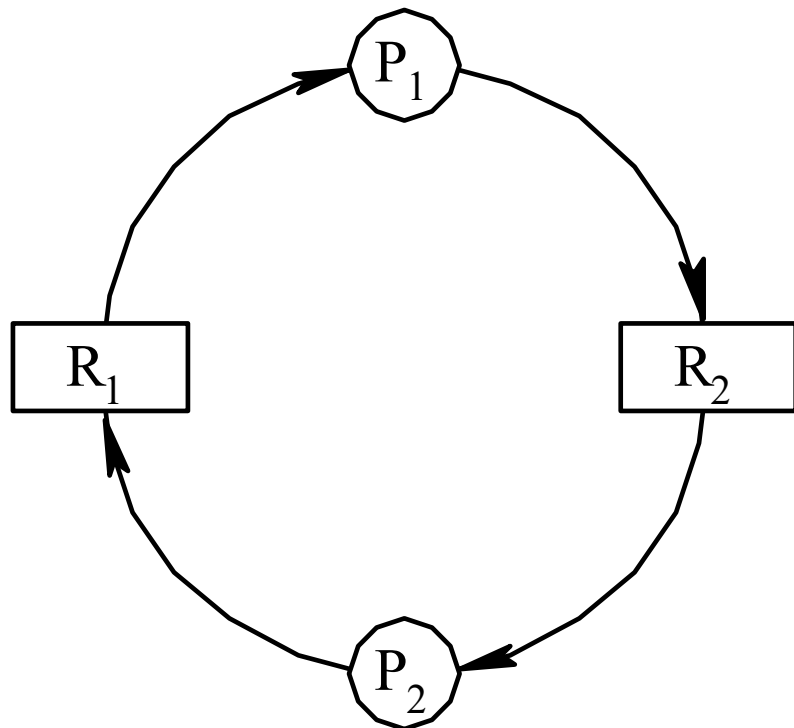
- **实时调度：开始截止时间、完成截止时间**
- **四种实时调度模型：**
- **最低松弛度优先算法：抢占式调度**
 - **松弛度 = 完成截止时间 - 需要运行时间 - 当前时间**
 - **在松弛度为0的时刻设置“地雷”，松弛度为0发生抢占**

产生死锁的原因和必要条件

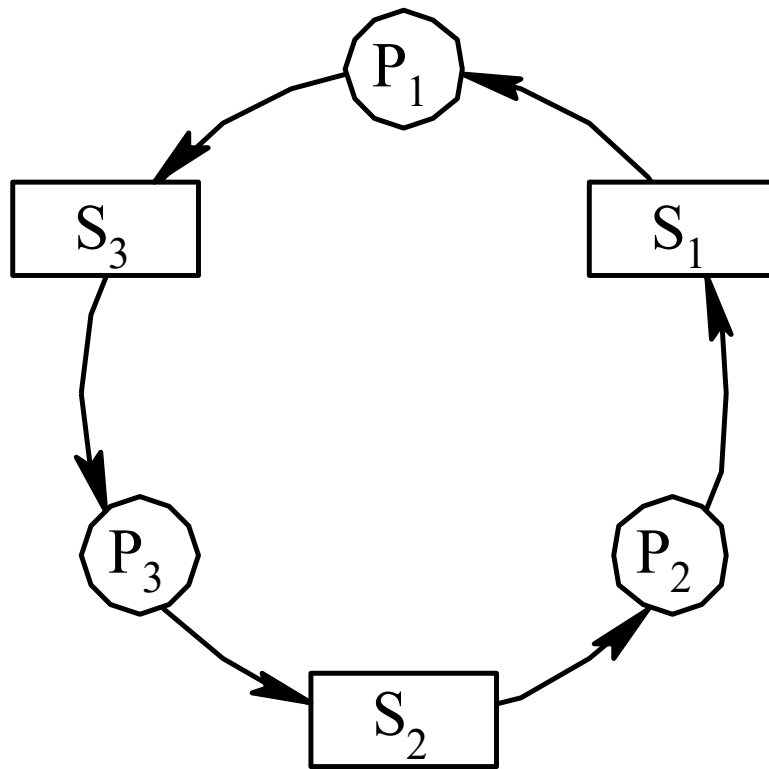
- 死锁：多个进程因竞争资源而造成的一种僵局，若无外力作用，这些进程都将永远不能再向前进
- 产生死锁的原因
 - 竞争资源
 - 进程间推进顺序非法

产生死锁的原因和必要条件

- 竞争资源引起进程死锁
 - 可剥夺和非剥夺性资源
 - 竞争非剥夺性资源
 - I/O设备共享死锁

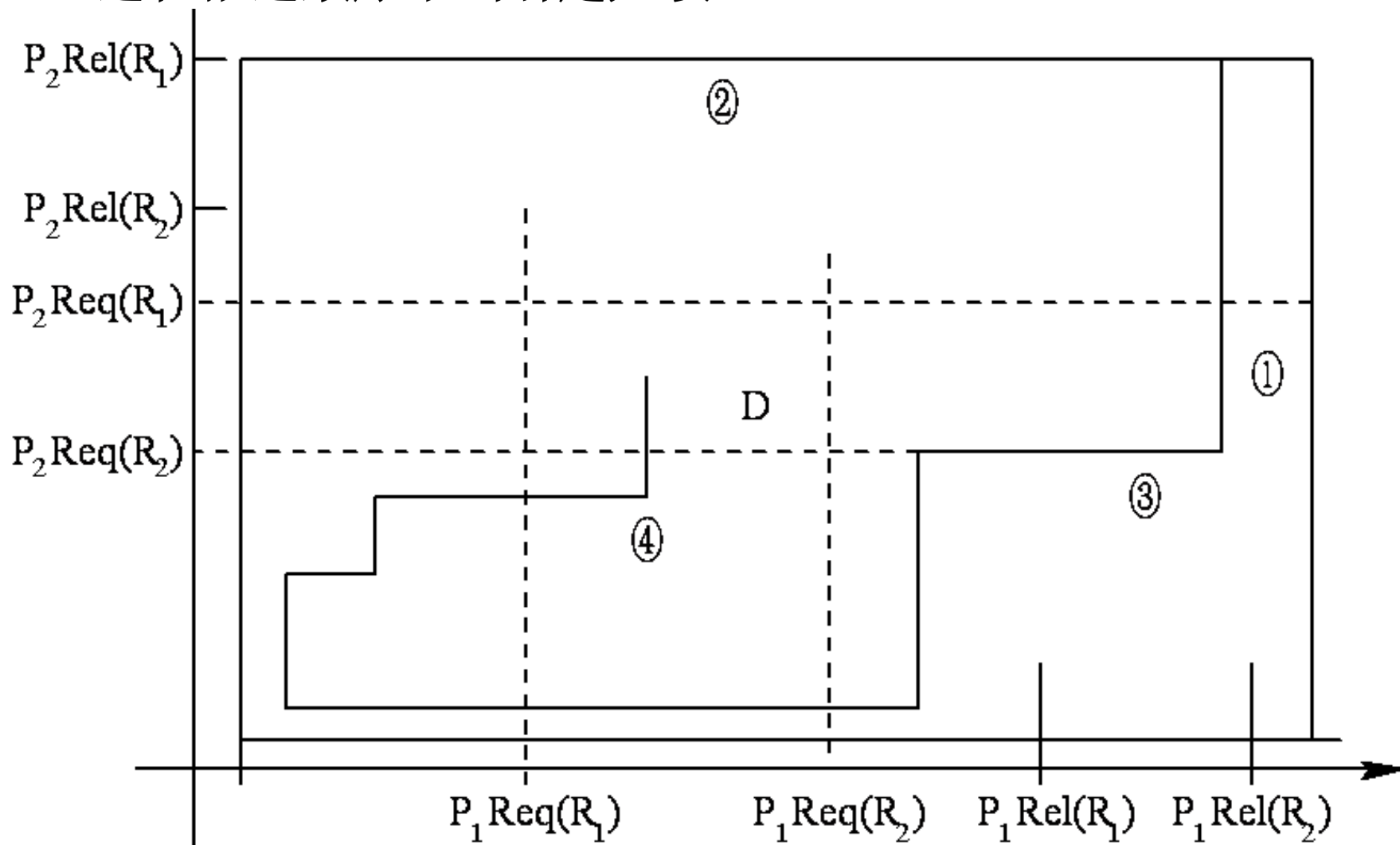


竞争临时性资源
进程通信死锁



产生死锁的原因和必要条件

■ 进程推进顺序不当引起死锁



产生死锁的原因和必要条件

■ 产生死锁的必要条件

- 互斥条件：请求资源为临界资源，将引发进程阻塞
- 请求和保持条件：持有旧资源，同时申请新资源
- 不剥夺条件：已获得资源在使用完之前，不被外力剥夺
- 环路等待条件：互相等待资源

产生死锁的原因和必要条件

■ 处理死锁的基本方法

- 预防死锁：设置限制条件，破坏产生死锁的一个或多个必要条件
- 避免死锁：在资源动态分配过程中，加入检查，防止系统进入不安全状态
- 检测死锁：建立检测机构检测死锁的发生和原因，确定相关的进程和资源
- 解除死锁：剥夺资源或撤销进程，从而解除死锁
- 检测死锁和解除死锁是一对合作关系

预防死锁的方法

- 预防死锁：破坏产生死锁的一个或多个必要条件
- 摒弃“请求和保持”条件
 - 资源一次性分配
 - 优点：简单，易于实现，安全
 - 缺点：资源浪费严重，进程延迟执行

预防死锁的方法

■ 摒弃“不剥夺”条件：

- 当资源请求不能满足，强行回收已分配资源
- 缺点：被剥夺资源的进程的工作全部作废，代价高
- 缺点：被剥夺资源的进程又申请资源，资源反复申请，性能下降

■ 摒弃“环路等待”条件

- 对资源排序，资源申请必须按照规定的顺序进行
- 缺点：资源排序要相对稳定
- 缺点：先获得的资源可能被长期闲置
- 缺点：对用户编程有限制

预防死锁的方法

■ 系统安全状态

- 所谓安全状态，是指系统能按某种进程顺序 $\langle P_1, P_2, \dots, P_n \rangle$ ，来为每个进程 P_i 分配其所需资源，直至满足每个进程对资源的最大需求，使每个进程都可顺利地完成任务，称 $\langle P_1, P_2, \dots, P_n \rangle$ 序列为安全序列。
- 如果系统无法找到这样一个安全序列，则称系统处于不安全状态。

进 程 \ 资 源 情 况	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3	3	3	2
P_1	3	2	2	2	0	0	1	2	2			
P_2	9	0	2	3	0	2	6	0	0			
P_3	2	2	2	2	1	1	0	1	1			
P_4	4	3	3	0	0	2	4	3	1			

预防死锁的方法

■ 数据结构定义

- **Max:** 进程对各种资源的最大需求
- **Allocation:** 进程已占用各种资源的数量
- **Need:** 进程对各种资源的剩余需求量
- **Max = Allocation + Need**
- **Available:** 系统当前可用资源数量
- **Work:** 每个进程完成后系统可用资源数量
 - 初始等于**Available**
- **Finish:** 每个进程能否完成

预防死锁的方法

■ T0时刻的安全序列

资源情况 进程	Work			Need			Allocation			Work + Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	
P ₁	3	3	2	1	2	2	2	0	0	5	3	2	true
P ₃	5	3	2	0	1	1	2	1	1	7	4	3	true
P ₄	7	4	3	4	3	1	0	0	2	7	4	5	true
P ₂	7	4	5	6	0	0	3	0	2	10	4	7	true
P ₀	10	4	7	7	4	3	0	1	0	10	5	7	true

3.6 预防死锁的方法

- 银行家算法应用背景：
 - 当前状态安全
 - 有新的资源分配请求
- 算法目的：判断新的请求是否安全
- 判断方法：
 - 虚拟执行这个分配请求，使得当前状态进入下一个状态
 - 在下一个状态中寻找安全序列
 - 若找到，就说明状态是安全，所以可以执行分配请求
 - 否则拒绝分配请求

预防死锁的方法

■ 执行流程:

- 前两步做合法性检查，后两步做尝试分配
- **Request_i**是进程**P_i**的请求向量
- **Request_i [j] = K**，表示进程**P_i**需要**K**个**j**类型的资源
- ① **If Request_i [j] ≤ Need [i,j] , continue;**
Else error; //因为所需资源数超过它所宣布的最大值
- ② **If Request_i [j] ≤ Available [j] , continue;**
Else wait; //尚无足够资源，**P_i**须等待

预防死锁的方法

■ 银行家算法执行流程:

- ③ 尝试把资源分配给进程 P_i ，修改下面数据结构的数值:

$\text{Available}[j] := \text{Available}[j] - \text{Request}_i[j]$;

$\text{Allocation}[i,j] := \text{Allocation}[i,j] + \text{Request}_i[j]$;

$\text{Need}[i,j] := \text{Need}[i,j] - \text{Request}_i[j]$;

- ④ 执行安全性算法，检查此次资源分配后，系统是否处于安全状态。

若安全，才正式将资源分配给进程 P_i ，完成本次分配；

否则， 将本次的试探分配作废，恢复原来的资源分配状态，让进程 P_i 等待。

3.6 预防死锁的方法

- 银行家算法之例：假定系统中有五个进程 {P₀, P₁, P₂, P₃, P₄} 和三类资源 {A, B, C}，各种资源的数量分别为10、5、7，在T₀时刻的资源分配情况如图 3-15 所示。

进 程 \ 资 源 情 况	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P ₀	7	5	3	0	1	0	7	4	3	3	3	2
										(2	3	0)
P ₁	3	2	2	2	0	0	1	2	2			
				(3	0	2)	(0	2	0)			
P ₂	9	0	2	3	0	2	6	0	0			
P ₃	2	2	2	2	1	1	0	1	1			
P ₄	4	3	3	0	0	2	4	3	1			

3.6 预防死锁的方法

■ T0时刻的安全性

资源情况 进程	Work			Need			Allocation			Work + Allocation			Finish
	A	B	C	A	B	C	A	B	C	A	B	C	
P ₁	3	3	2	1	2	2	2	0	0	5	3	2	true
P ₃	5	3	2	0	1	1	2	1	1	7	4	3	true
P ₄	7	4	3	4	3	1	0	0	2	7	4	5	true
P ₂	7	4	5	6	0	0	3	0	2	10	4	7	true
P ₀	10	4	7	7	4	3	0	1	0	10	5	7	true

3.6 预防死锁的方法

- **P1请求资源：P1发出请求向量Request1(1, 0, 2)，系统按银行家算法进行检查：**
 - ① **$\text{Request1}(1, 0, 2) \leq \text{Need1}(1, 2, 2)$**
 - ② **$\text{Request1}(1, 0, 2) \leq \text{Available1}(3, 3, 2)$**
 - ③ 系统先假定可为P1分配资源，并修改**Available**, **Allocation1**和**Need1**向量，由此形成的资源变化情况如图 3-15 中的圆括号所示。 •
 - ④ 再利用安全性算法检查此时系统是否安全。

3.6 预防死锁的方法

进程P1请求分配资源Request (1, 0, 2)后:

资源 进程	Work			Need			Allocation			Work+Allocation			完成
	A	B	C	A	B	C	A	B	C	A	B	C	
P1	2	3	0	0	2	0	3	0	2	5	3	2	true
P3	5	3	2	0	1	1	2	1	1	7	4	3	true
P4	7	4	3	4	3	1	0	0	2	7	4	5	true
P0	7	4	5	7	4	3	0	1	0	7	5	5	true
P2	7	5	5	6	0	0	3	0	2	10	5	7	true

安全进程执行序列P1→ P3→ P4→ P0→ P2

3.6 预防死锁的方法

- **P4请求资源：** P4发出请求向量 $\text{Request}_4(3, 3, 0)$ ，系统按银行家算法进行检查：
 - ① $\text{Request}_4(3, 3, 0) \leq \text{Need}_4(4, 3, 1)$;
 - ② $\text{Request}_4(3, 3, 0) \geq \text{Available}(2, 3, 0)$ ，让P4等待。

- **P0请求资源：** P0发出请求向量 $\text{Request}_0(0, 2, 0)$ ，系统按银行家算法进行检查：
 - ① $\text{Request}_0(0, 2, 0) \leq \text{Need}_0(7, 4, 3)$;
 - ② $\text{Request}_0(0, 2, 0) \leq \text{Available}(2, 3, 0)$;
 - ③ 系统暂时先假定可为P0分配资源，并修改有关数据，如图 所示。

资源情况 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	7	5	3	0	1	0	7	4	3	2	3	0
P1	3	2	2	3	0	2	0	2	0			
P2	9	0	2	3	0	2	6	0	0			
P3	2	2	2	2	1	1	0	1	1			
P4	4	3	3	0	0	2	4	3	1			

资源情况 进程	Max			Allocation			Need			Available		
	A	B	C	A	B	C	A	B	C	A	B	C
P0	7	5	3	0	3	0	7	2	3	2	1	0
P1	3	2	2	3	0	2	0	2	0			
P2	9	0	2	3	0	2	6	0	0			
P3	2	2	2	2	1	1	0	1	1			
P4	4	3	3	0	0	2	4	3	1			

3.6 预防死锁的方法

进程P0请求分配资源Request (0, 2, 0)后, 执行安全性检查:

资源 进程	Work			Need			Allocation			Work+Allocation			完成
	A	B	C	A	B	C	A	B	C	A	B	C	
P0	2	1	0	7	2	3	0	3	0				false
P1				0	2	0	3	0	2				false
P2				6	0	0	3	0	2				false
P3				0	1	1	2	1	1				false
P4				4	3	1	0	0	2				false

不存在安全进程执行序列, 预分配无效, P0等待

3.6 预防死锁的方法

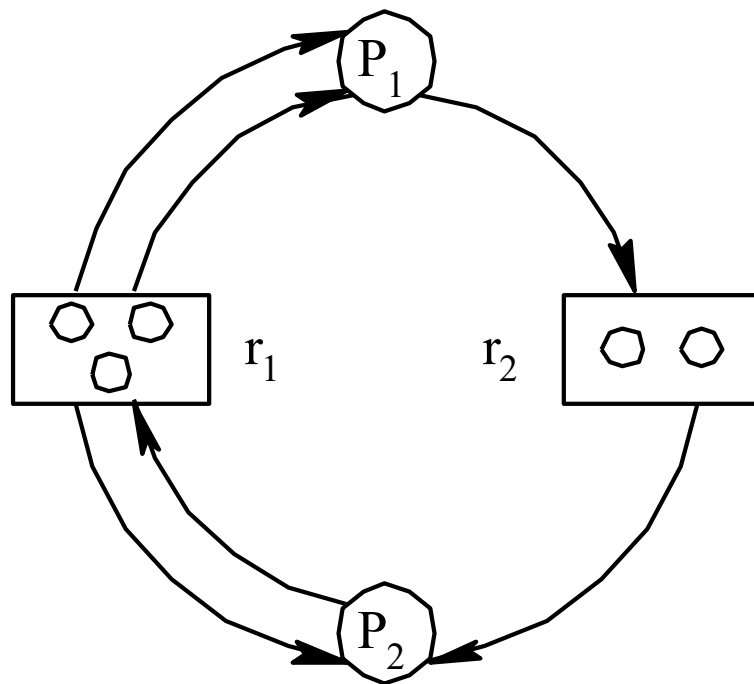
■ 银行家算法的缺点：

- 很少进程能够在运行前知道它所需资源的最大值
- 系统内的进程数是不固定的，往往在不断变化，有新的进入，有完成的退出
- 可用资源可能突然发生故障变为不可用，例如磁带机、硬盘读写故障
- 算法开销大，实时性不好

3.7 死锁的检测与解除

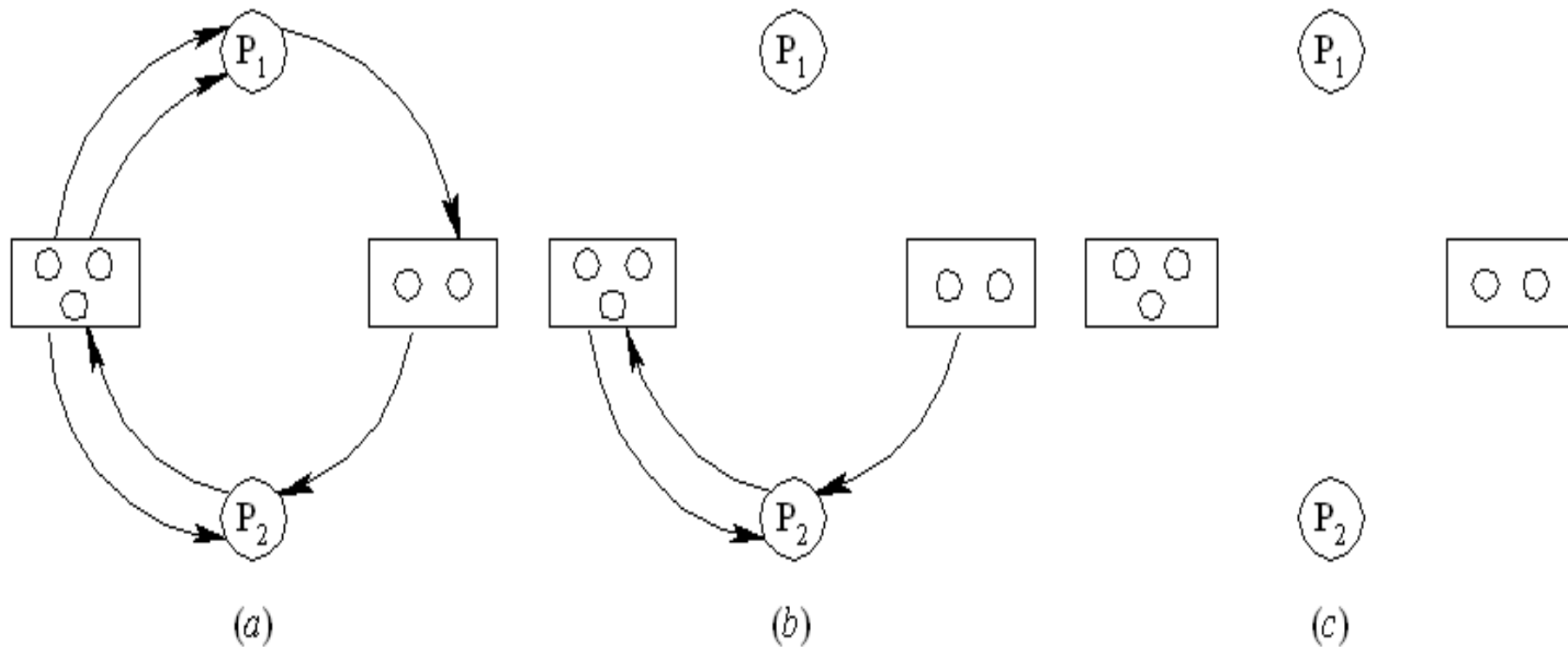
3.7.1 死锁的检测

- 资源分配图
- 结点表示进程
- 方框表示资源
- 资源数量等于方框内圆圈数量
- 进程占有资源：资源指向结点的有向边
- 进程请求资源：结点指向资源的有向边
- 占有或请求数量等于边的数量



3.7 死锁的检测与解除

- 死锁定理：资源分配不可完全简化，则处于死锁状态



3.7 死锁的检测与解除

■ 死锁的解除：剥夺资源、撤消进程

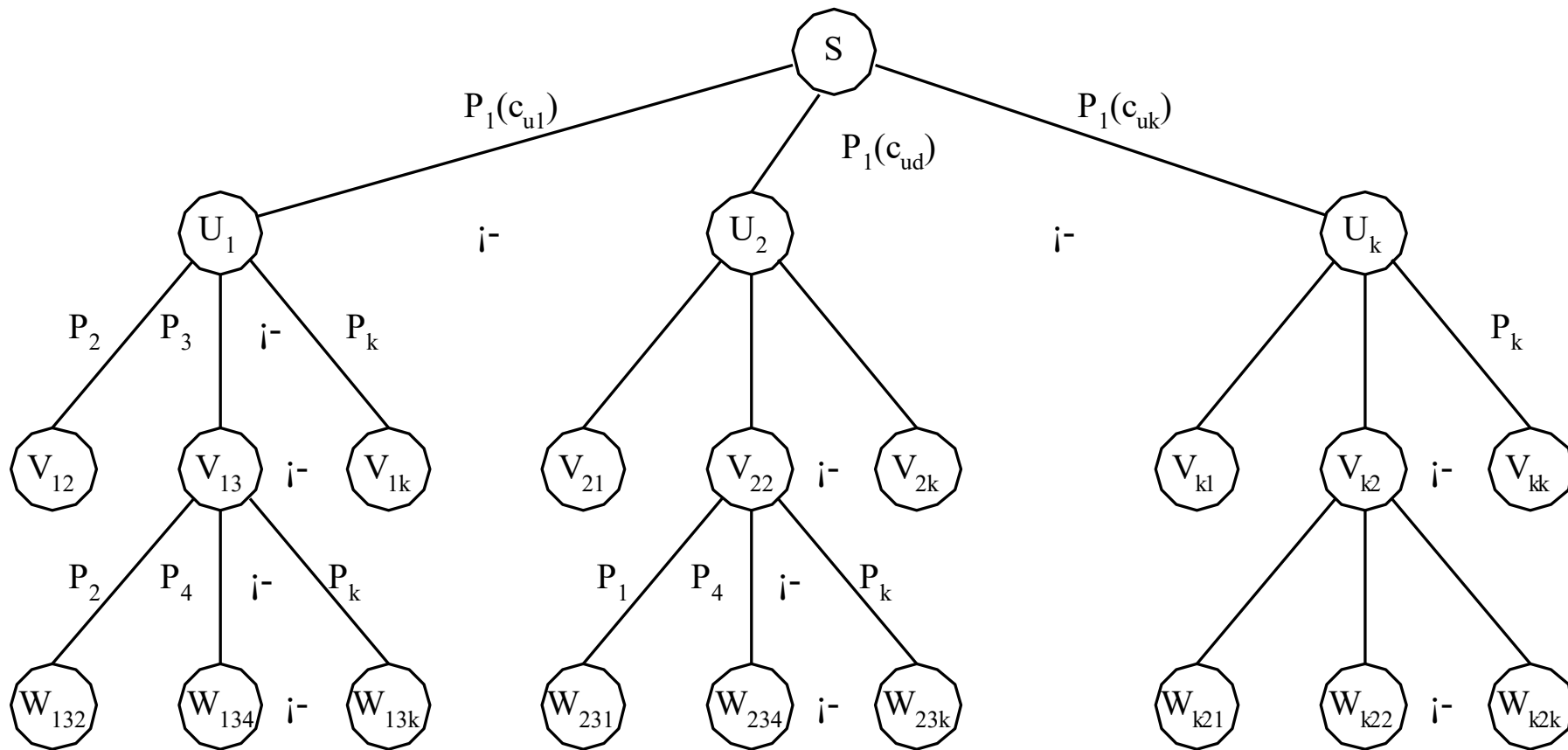


图 3-21 付出代价最小的死锁解除方法